

Research Report: Autonomous Code Migration & Modernization Engine

Introduction

- AI-powered platform designed for software modernization and legacy code migration.
- Helps transform outdated systems such as:
 - Python 2 → Python 3
 - Java 8 → Java 21
 - Monolithic architecture → Microservices
- Reduces manual coding effort, migration cost, and development time.
- Uses Artificial Intelligence, static code analysis, dependency graphs, and automated testing to modernize applications safely.

Objective

- Build an intelligent migration system capable of:
 - Understanding source code semantically
 - Detecting deprecated or risky patterns
 - Generating migration plans
 - Automatically transforming code
 - Validating transformed code
 - Supporting human review workflows
- Preserve business logic while modernizing enterprise applications.

Technologies Used

- **Frontend:** Streamlit / React
- **Backend:** Python, FastAPI
- **Code Parsing:** Tree-sitter, Python AST
- **AI Models:** DeepSeek-Coder, CodeLlama, Ollama
- **Dependency Analysis:** NetworkX, Graph analysis libraries
- **Database:** SQLite / PostgreSQL
- **DevOps & Deployment:** Docker, GitHub, CI/CD pipelines

Existing Market Solutions

- Existing enterprise modernization tools include:
 - AWS Transform
 - IBM Application Modernization
 - Fujitsu Application Transform

- Code Metal AI
- Most existing tools are:
 - Expensive
 - Enterprise-restricted
 - Limited to specific migration types
- Proposed system improves flexibility by combining:
 - AI-driven transformation
 - Dependency-aware analysis
 - Automated validation
 - Rollback mechanisms
 - Human review workflows

Working Methodology

- Repository upload and project scanning
- AST parsing and dependency graph generation
- Migration pattern detection
- AI-powered code transformation
- Validation using:
 - Syntax checks
 - Type analysis
 - Automated testing
- Diff review and rollback support
- Report generation and deployment support
- Follows a “Trust but Verify” architecture to ensure migration reliability.

Real-World Applications

- Useful for:
 - Banking systems
 - Healthcare platforms
 - Government software
 - Telecom infrastructure
 - SaaS companies
 - FinTech applications
- Helps organizations:
 - Reduce technical debt
 - Improve scalability

- Enable cloud migration
- Improve maintainability
- Reduce operational risks

Conclusion

- Represents an advanced AI-driven approach for enterprise software modernization.
- Combines:
 - Large Language Models
 - Dependency graph analysis
 - Static code analysis
 - Automated testing
 - Intelligent orchestration
- Has strong potential for future enterprise adoption and autonomous software engineering systems.